



ML Projects — Tonye Maxwell Kentebe

BASc Computer Engineering · University of Waterloo · GPA 3.9/4.0



Velocity Co-op Problem Award ·



President's Scholarship



tmkenteb@uwaterloo.ca ·



tonye-kentebe.github.io

Six end-to-end machine learning projects covering the full breadth of modern ML — from classical regression and NLP to deep learning and anomaly detection. Each project is a fully documented Jupyter notebook with EDA, model training, evaluation, and visualisations.



Portfolio website: tonye-kentebe.github.io



Projects



Machine Learning Notebooks

#	Notebook	Domain	Key Result
01	House Price Prediction	Regression	$R^2 \approx 0.82$ (Gradient Boosting)
02	Customer Churn Classification	Binary Classification	ROC-AUC ≈ 0.94
03	Customer Segmentation	Unsupervised / Clustering	4 clean segments, Silhouette ≈ 0.62
04	Sentiment Analysis	NLP	98% accuracy, AUC ≈ 0.99
05	Fraud Detection	Anomaly Detection	Avg Precision ≈ 0.92
06	Image Classification CNN	Deep Learning / CV	$\sim 88\%$ accuracy on CIFAR-10

Other Projects

Project	Description	Tech
Game Time Tracker	Desktop app that tracks your gaming sessions and alerts you when you've hit your time limit	Python · Tkinter

Project Summaries

01 · House Price Prediction

Predicts median California house prices using a full regression pipeline.

- **EDA** with correlation analysis and scatter plots
 - **Feature engineering**: rooms-per-person, income-per-room, outlier clipping
 - **Models compared**: Linear Regression, Ridge, Lasso, Random Forest, Gradient Boosting
 - **Best result**: $R^2 \approx 0.82$, RMSE ≈ 0.48 (Gradient Boosting)
 - **Key finding**: **MedInc** (median income) is by far the strongest predictor
-

02 · Customer Churn Classification

Predicts whether a telecom customer will churn from their subscription.

- **Class imbalance** handled via `class_weight='balanced'` (SMOTE-ready)
 - **Models**: Logistic Regression, Random Forest, Gradient Boosting
 - **Best result**: ROC-AUC ≈ 0.94
 - **Business-aware threshold tuning** — lowering the decision threshold recovers more churners
 - **Key finding**: Month-to-month contracts + high charges + low tenure = highest risk
-

03 · Customer Segmentation

Groups e-commerce customers into meaningful segments using RFM (Recency, Frequency, Monetary) features.

- **Elbow method + Silhouette + Davies-Bouldin** to find optimal $k=4$
 - **K-Means** and **DBSCAN** both implemented and compared
 - **PCA** reduces to 2D — explains 90%+ of variance
 - **Segments identified:** Champions, Loyal, At Risk, Lost/Hibernating
 - **Business actions** derived for each segment
-

04 · Sentiment Analysis (NLP)

Classifies movie reviews as positive or negative using text pipelines.

- **Text preprocessing:** lowercasing, regex cleaning, stopword removal
 - **Vectorisation:** Bag-of-Words, TF-IDF with unigrams, bigrams, and character n-grams
 - **Models:** Naive Bayes, Logistic Regression, LinearSVC
 - **Best result:** 98% accuracy, AUC ≈ 0.99 (Logistic Regression + TF-IDF bigrams)
 - **Error analysis** on misclassified examples + most-informative features extracted
-

05 · Credit Card Fraud Detection

Detects fraudulent transactions in a highly imbalanced dataset (0.5% fraud rate).

- **Why standard accuracy fails:** a model predicting "no fraud" always gets 99.5% accuracy
 - **Isolation Forest** (unsupervised) vs. supervised classifiers
 - **Best result:** Average Precision ≈ 0.92 (Gradient Boosting)
 - **Cost-sensitive threshold tuning:** minimises total business cost (FN cost $\gg$ FP cost)
 - **Key finding:** Anomalous V1, V2, V4, V10, V12 features + off-hour transactions = fraud signals
-

06 · Image Classification with CNN

Trains a convolutional neural network to classify images into 10 categories.

- **Dataset:** CIFAR-10 — 60,000 32×32 colour images, 10 classes
 - **Architecture:** 3 Conv blocks with BatchNorm, MaxPooling, and Dropout
 - **Data augmentation:** horizontal flip, rotation, zoom, shift
 - **Custom CNN result:** ~82% test accuracy
 - **Transfer Learning** with MobileNetV2 backbone: ~88% test accuracy
 - **Analysis:** confusion matrix, per-class accuracy, misclassified image visualisation
-

Game Time Tracker

A desktop Python application that helps you manage your gaming sessions.

- **Set any game name** and a custom time limit (hours + minutes)
- **Live timer** with a colour-coded progress bar — green → yellow → red as you approach your limit
- **Popup alerts** at 75%, 90%, and when your limit is hit (window flashes to grab your attention)
- **Pause / resume** support for breaks
- **Session history** saved to `game_sessions.json` — tracks every session across days
- **No external dependencies** — runs with Python's built-in `tkinter`

`python game_time_tracker.py`

Tech Stack

Languages: Python 3.10

ML / Data: scikit-learn · TensorFlow · Keras · NumPy · Pandas

Visualisation: Matplotlib · Seaborn

NLP: TF-IDF · CountVectorizer · scikit-learn Pipelines

Deep Learning: Keras CNN · MobileNetV2 · ImageDataGenerator

GUI: Tkinter (Game Time Tracker)

Environment: Jupyter Notebook

How to Run

1. Clone the repo

```
git clone https://github.com/tonye-kentebe/ml-projects.git
```

```
cd ml-projects
```

2. Install dependencies

```
pip install numpy pandas matplotlib seaborn scikit-learn tensorflow jupyter
```

3. Launch Jupyter

```
jupyter notebook
```

4. Open any notebook — each one is fully self-contained and runs top to bottom.

No GPU required for notebooks 01–05. Notebook 06 (CNN) benefits from a GPU but runs on CPU too — training will just take longer.

To run the Game Time Tracker (no install needed — uses Python built-ins):

```
python game_time_tracker.py
```

Contact

Tonye Maxwell Kentebe

BASc Computer Engineering · University of Waterloo

 tmkenteb@uwaterloo.ca

 tonye-kentebe.github.io

 github.com/tonye-kentebe

Built as part of a personal ML portfolio — all datasets are either built-in sklearn datasets or synthetically generated.